전이 학습 코드 해설

남이 학습한 모델 빌려 쓰기 · 5일차 21~25차시 · 예제 10-transfer-learning

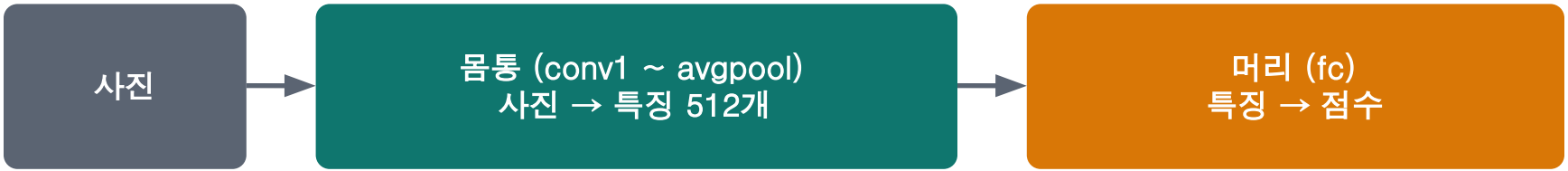
오늘 볼 코드 24개

차시	파일	핵심 질문
21	b01 불러오기 · b02 목록 · b03 몸통과 머리 · b04 전처리 · b05 ImageNet 예측 · b06 EuroSAT · 전이도구 · b07 분할	남의 모델을 어떻게 가져와 들여다보나?
22	b08 얼리기 · b09 특징 뽑아 두기 · b10 BN 함정 · b11 전처리 실수 · b12 입력 크기	눈은 그대로, 판단만 새로 배우려면?
23	b13 처음부터 · b14 미세조정 학습률 · b15 층별 학습률 · b16 어디까지 풀까 · b17 무엇이 바뀌었나	몸통까지 조금씩 고치면?
24	b18 머리 바꾸기 일반화 · b19 다른 몸통 · b23 흑백 입력 · b20 저장·불러오기	다른 모델, 다른 입력에도 쓰려면?
25	b21 데이터 양별 비교 · b22 시험 데이터로 최종 확인	세 방법 중 언제 무엇을?

진행 방식

코드 해설 → 차시 끝에 문제, 문제 바로 다음 장에 정답
학습 결과 숫자는 강의용 컴퓨터(MPS) 기준이라 컴퓨터마다 조금 다를 수 있다

전이 학습 한 장 요약: 눈은 빌리고 판단만 새로



방법	몸통	머리	사진 500장 검증	차시
처음부터 학습	무작위에서 전부 학습	전부 학습	68.1%	23
특징 추출	ImageNet 가중치 그대로 (얼림)	새로 학습	**87.0%**	22
미세조정	ImageNet 가중치에서 조금씩 고침	새로 학습	**93%대**	23

같은 ResNet18 구조, 같은 사진 500장. 출발점만 달라도 20%p 넘게 차이 난다

21 사전학습 모델과 이 장의 데이터

b01 ~ b07 · 전이도구.py · 책 10장 들어가며 ~ 10.3

b01 사전학습 모델 불러오기: 한 줄로 1,170만 개

```
가중치 = models.ResNet18_Weights.DEFAULT    # ImageNet으로 학습한 가중치
모델 = models.resnet18(weights=가중치)       # 처음 한 번 약 45MB 다운로드

개수 = sum(p.numel() for p in 모델.parameters())
바이트 = sum(p.numel() * p.element_size()
             for p in 모델.parameters())
print(f"파라미터 {개수:,}개, float32로 {바이트/1024**2:.1f}MB")
print(가중치.meta["_metrics"]["ImageNet-1K"]["acc@1"])
범주 = 가중치.meta["categories"]            # 100까지 이름
```

```
ResNet18_Weights.IMAGENET1K_V1
파라미터 11,689,512개, float32로 44.6MB
모델 카드에 적힌 개수 11,689,512개, 파일 44.661MB
ImageNet 검증 데이터 정확도(top-1): 69.758 %
알아보는 범주 1000가지: ['tench', 'goldfish', ...] ... ['ear', 'toilet tissue']
```

weights= 가 핵심

weights=None: 무작위 가중치 (구조만)
weights=DEFAULT: 가장 좋은 학습 결과
같은 함수, 인자 하나로 출발점이 달라진다

element_size()

숫자 하나의 바이트 수
float32 = 4바이트 → 1,170만 × 4 ≈ 44.6MB

meta = 모델 카드

파라미터 수, 파일 크기, ImageNet 정확도, 범주 이름이 들어 있다
내려받기 전에도 읽을 수 있다

b02 어떤 모델이 있을까: 모델 카드만 읽기

```
이름들 = models.list_models(module=models)    #사진 분류 모델 목록
print(f"사진 분류 모델 {len(이름들)}종:", 이름들[:4], "...")

for 이름 in ["mobilenet_v3_small", "resnet18",
             "efficientnet_b0", "resnet50", "vit_b_16"]:
    w = models.get_model_weights(이름).DEFAULT  # 정보만, 다운로드 X
    정확도 = w.meta["_metrics"]["ImageNet-1K"]["acc@1"]
    print(f"{이름:19} 파라미터 {w.meta['num_params']/1e6:5.1f}백만, "
          f"파일 {w.meta['_file_size']:6.1f}MB, top-1 정확도:.1f}%")
```

```
mobilenet_v3_small  파라미터   2.5백만, 파일    9.8MB, top-1 67.7%
resnet18            파라미터  11.7백만, 파일   44.7MB, top-1 69.8%
efficientnet_b0     파라미터   5.3백만, 파일   20.5MB, top-1 77.7%
resnet50            파라미터  25.6백만, 파일   97.8MB, top-1 80.9%
vit_b_16            파라미터  86.6백만, 파일  330.3MB, top-1 81.1%
```

get_model_weights(이름)

이름 문자열로 가중치 정보를 찾는다
.DEFAULT = 그 모델의 대표 가중치

표 읽는 법

크기와 정확도는 비례하지 않는다
EfficientNet-B0: ResNet18의 절반 크기인데 8%p 높음

주의

ImageNet 순위가 우리 데이터에서도 그대로라는 보장은 없다 (24차시 b19에서 확인)

b03 몸통과 머리: 층을 하나씩 통과시키기

```
모델 = models.resnet18(weights=None)    #구조만 볼 땐 가중치 불필요
x = torch.zeros(1, 3, 224, 224)         #사진 1장, 3색, 224×224
for 이름, 부품 in 모델.named_children():
    if 이름 == "fc":
        x = torch.flatten(x, 1)  # forward가 fc 앞에서 하는 일
    x = 부품(x)
    개수 = sum(p.numel() for p in 부품.parameters())
    print(f"{이름:8} {type(부품).__name__:17} {개수:>10,}개 → {tuple(x.shape)}")
print(모델.fc)
```

```
conv1    Conv2d                9,408개 → (1, 64, 112, 112)
bn1      BatchNorm2d             128개 → (1, 64, 112, 112)
maxpool  MaxPool2d                 0개 → (1, 64, 56, 56)
layer1   Sequential          147,968개 → (1, 64, 56, 56)
layer2   Sequential          525,568개 → (1, 128, 28, 28)
layer3   Sequential        2,099,712개 → (1, 256, 14, 14)
layer4   Sequential        8,393,728개 → (1, 512, 7, 7)
avgpool  AdaptiveAvgPool2d         0개 → (1, 512, 1, 1)
fc       Linear              513,000개 → (1, 1000)
```

named_children()

모델 바로 아래 부품을 (이름, 부품)으로 하나씩 꺼낸다
9장 b09처럼 모양을 따라가는 방법

몸통 = conv1 ~ avgpool

사진을 특징 512개로 바꾸는 눈
다른 문제에도 그대로 쓸모 있다

머리 = fc

512 × 1000 + 1000 = 513,000개
ImageNet 1000범주용 → 우리 문제(10가지)에 맞게 새로 바꾼다

b04 전처리 맞추기: 모델이 배운 기준 그대로

```
전처리 = models.ResNet18_Weights.DEFAULT.transforms()
print(전처리)
사진, 정답 = datasets.EuroSAT("data", download=True)[0]
x = 전처리(사진)                          # (3, 224, 224) float32

# 같은 일을 v2 변환으로 풀어 쓰면
풀어쓰기 = v2.Compose([
    v2.ToImage(),
    v2.Resize(256),                       #짧은 변을 256으로
    v2.CenterCrop(224),                   #가운데 224×224
    v2.ToDtype(torch.float32, scale=True),   # 0~255 → 0~1
    v2.Normalize(mean=[0.485, 0.456, 0.406],
                 std=[0.229, 0.224, 0.225]),
])
print(torch.allclose(풀어쓰기(사진), x, atol=1e-5))
```

```
ImageClassification( crop_size=[224] resize_size=[256]
  mean=[0.485, 0.456, 0.406] std=[0.229, 0.224, 0.225] ... )
원본: (64, 64) RGB
전처리 후: (3, 224, 224) torch.float32
풀어 쓴 것과 같은가: True
```

weights.transforms()

그 가중치를 학습할 때 쓴 전처리를 그대로 돌려준다
print해서 내용을 꼭 확인

mean·std는 ImageNet 값

모델은 이 눈금으로 정규화된 사진만 봐 왔다
우리 데이터가 위성 사진이어도 이 값으로 (22차시 b11에서 숫자로 확인)

크기는 바꿔도 된다

64×64 → 256 → 224로 키운 것
이 장은 계산을 아끼려고 112를 쓴다 (b12)

b05 ImageNet의 눈으로 위성 사진 보기

```
가중치 = models.ResNet18_Weights.DEFAULT
모델 = models.resnet18(weights=가중치).eval()
전처리 = 가중치.transforms()
범주 = 가중치.meta["categories"]
원본 = datasets.EuroSAT("data", download=True)
정답들 = torch.tensor(원본.targets)
for k in [1, 3, 7, 8, 9]:          #숲, 도로, 주거지, 강, 바다·호수
    i = (정답들 == k).nonzero()[0].item()    #그 클래스 첫 사진
    with torch.no_grad():
        확률 = 모델(전처리(원본[i][0]).unsqueeze(0)).softmax(dim=1)[0]
    위 = 확률.topk(3)
    print(원본.classes[k], [(범주[j], f"{p:.0%}")
                            for p, j in zip(위.values, 위.indices)])
```

```
Forest      → dugong 6%, velvet 4%, Petri dish 3%
Highway     → safety pin 11%, hand-held computer 10%, reel 4%
Residential → handkerchief 13%, stole 6%, terrapin 6%
River       → Petri dish 4%, Windsor tie 3%, hand-held computer 3%
SeaLake     → space shuttle 3%, nematode 3%, flagpole 3%
```

줄마다 읽기

nonzero()[0]: 정답이 k인 첫 번호
unsqueeze(0): 사진 1장을 (1, 3, H, W) 묶음으로
softmax → topk(3): 확률 상위 3개

결과 해석

"숲"을 듀공, 벨벳이라고 한다
확률도 3~13%로 자신이 없다

결론

눈(몸통)은 좋지만 우리 문제의 단어를 모른다
→ 몸통은 남기고 머리만 다시 가르친다

b06 EuroSAT: 64×64 위성 사진 27,000장

```
원본 = datasets.EuroSAT("data", download=True)    #약 90MB
정답들 = torch.tensor(원본.targets)
print(f"사진 {len(원본):,}장, 크기 {원본[0][0].size}")
for k, (이름, 개수) in enumerate(
        zip(원본.classes, torch.bincount(정답들).tolist())):
    # 클래스마다 앞 100장의 평균 색 (0~255)
    번호 = (정답들 == k).nonzero().flatten()[:100].tolist()
    평균색 = torch.stack([v2.functional.pil_to_tensor(원본[i][0])
                          for i in번호]).float().mean(dim=(0, 2, 3))
    print(f"{k} {이름:20} {개수:,}장 평균 색 R·G·B = "
          f"{평균색.round().int().tolist()}")
```

```
사진 27,000장, 크기 (64, 64)
1 Forest              3,000장 평균 색 R·G·B = [39, 64, 76]
9 SeaLake             3,000장 평균 색 R·G·B = [45, 69, 86]
3 Highway             2,500장 평균 색 R·G·B = [89, 97, 103]
7 Residential 3,000장 평균 색 R·G·B = [95, 101, 110]
2 HerbaceousVegetation 3,000장 [100, 99, 104]
(일부만, 순서는 PDF 표 기준)
```

열 가지 땅

0 경작지 · 1 숲 · 2 초지 · 3 고속도로 · 4 산업지역 · 5 목초지 · 6 과수원 · 7 주거지 · 8 강 · 9 바다·호수

코드 포인트

bincount: 클래스별 장 수 (2,000~3,000장)
pil_to_tensor: 사진 → (3, 64, 64) uint8
mean(dim=(0, 2, 3)): 장·세로·가로 평균 → 채널 3개

색만으로는 안 된다

숲과 바다·호수, 고속도로·주거지·초지는 평균 색이 거의 같다
→ 무늬와 모양으로 가려야 한다

전이도구.py ①: 데이터 불러오기와 변환

```
IMAGENET_평균 = [0.485, 0.456, 0.406]
IMAGENET_표준편차 = [0.229, 0.224, 0.225]
구간_시작 = {"학습": 0, "검증": 1000, "시험": 1500}

def 유로샛_불러오기(구간, 클래스당):
    원본 = datasets.EuroSAT("data", download=True)
    정답들 = torch.tensor(원본.targets)
    번호 = []
    for k in range(len(원본.classes)):
        후보 = (정답들 == k).nonzero().flatten()
        섞기 = torch.randperm(len(후보),
                  generator=torch.Generator().manual_seed(k))
        시작 = 구간_시작[구간]
        번호 += 후보[섞기[시작:시작 + 클래스당]].tolist()
    사진 = torch.stack([v2.functional.pil_to_tensor(원본[i][0])
                        for i in 번호])
    return 사진, 정답들[번호]

def 변환_만들기(크기=112, 증강=False):
    단계 = [v2.Resize(크기)]
    if 증강:   # 위성 사진은 위아래가 없어 상하 뒤집기도 OK
        단계 += [v2.RandomHorizontalFlip(), v2.RandomVerticalFlip()]
    단계 += [v2.ToDtype(torch.float32, scale=True),
             v2.Normalize(IMAGENET_평균, IMAGENET_표준편차)]
    return v2.Compose(단계)
```

유로샛_불러오기

클래스마다 씨앗 k로 섞은 순서를 만들고 학습은 0번째부터, 검증은 1000번째부터, 시험은 1500번째부터 잘라 쓴다
→ 같은 사진이 두 구간에 들어갈 수 없다

왜 클래스마다 같은 수?

클래스별 장 수가 2,000~3,000으로 달라서 같은 수를 뽑아 공정하게 비교 (11차시 분할)

변환_만들기

크기 맞추기 → (뒤집기) → 0~1 → ImageNet 정규화
학습용만 증강=True

전이도구.py ②: 학습·평가·특징 뽑기

```
class 사진_데이터셋(Dataset):
    def __getitem__(self, i):   # 꺼낼 때마다 변환 (증강이 매번 새로)
        return self.변환(self.사진[i]), self.정답[i]

def 로더_만들기(사진, 정답, 증강=False, 섞기=False, 크기=112):
    데이터셋 = 사진_데이터셋(사진, 정답, 변환_만들기(크기, 증강))
    return DataLoader(데이터셋, batch_size=64, shuffle=섞기,
                      generator=torch.Generator().manual_seed(0))

@torch.no_grad()
def 특징_뽑기(몸통, 로더, 장치):
    몸통.eval()
    특징, 정답 = zip(*[(몸통(x.to(장치)).cpu(), y) for x, y in 로더])
    return torch.cat(특징), torch.cat(정답)

def 머리_학습(특징, 정답, 에폭=100):
    torch.manual_seed(0)
    머리 = nn.Linear(특징.shape[1], int(정답.max()) + 1)
    옵티마이저 = torch.optim.Adam(머리.parameters(), lr=1e-3)
    for _ in range(에폭):           # 데이터가 작아 한 번에 전부
        옵티마이저.zero_grad()
        nn.functional.cross_entropy(머리(특징), 정답).backward()
        옵티마이저.step()
    return 머리
```

한_에폭_학습 · 정확도

6장부터 써 온 표준 학습 루프·평가 루프 그대로 (화면에서는 생략)

특징_뽑기

얼린 몸통에 사진을 한 번씩 통과시켜 (장수, 512) 특징을 모은다
zip(*[...]): (특징, 정답) 쌍 목록을 특징 목록과 정답 목록으로 나눔

머리_학습

특징만 가지고 Linear 하나를 100에폭 학습
출력 수 = 정답 최댓값 + 1 = 10

b07 학습·검증·시험으로 나눠 불러오기

```
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 50)
검증_사진, 검증_정답 = 유로샛_불러오기("검증", 100)
시험_사진, 시험_정답 = 유로샛_불러오기("시험", 100)
print(f"학습 사진이 차지하는 메모리 "
      f"{학습_사진.numel() * 학습_사진.element_size() / 1024**2:.1f}MB")

x, y = next(iter(로더_만들기(학습_사진, 학습_정답,
                            증강=True, 섞기=True)))
print("배치 하나:", tuple(x.shape), x.dtype, y[:8].tolist())
print("채널별 평균:", x.mean(dim=(0, 2, 3)),
      "표준편차:", x.std(dim=(0, 2, 3)))
```

```
학습: (500, 3, 64, 64) torch.uint8, 클래스별 [50, 50, ..., 50]
검증: (1000, 3, 64, 64) torch.uint8, 클래스별 [100, ..., 100]
시험: (1000, 3, 64, 64) torch.uint8, 클래스별 [100, ..., 100]
학습 사진이 차지하는 메모리 5.9MB
배치 하나: (64, 3, 112, 112) torch.float32 [8, 9, 8, 8, 9, 5, 4, 3]
채널별 평균: ['-0.66', '-0.37', '-0.05'] 표준편차: ['0.86', '0.55', '0.44']
```

27,000장 중 500장만

정답 붙은 사진이 몇백 장뿐인 현장을 흉내 냄
이게 전이 학습이 빛나는 상황

uint8로 보관

사진은 0~255 정수로 메모리에 두고 꺼낼 때 변환
500 × 3 × 64 × 64 × 1바이트 = 5.9MB

평균이 0이 아니다?

ImageNet 눈금으로 정규화했기 때문
위성 사진은 ImageNet보다 어두워 음수 쪽으로 치우침 → 틀린 게 아니다 (b11)

21차시 문제

1 **입력을 160×160으로** [b03] 기본

b03의 가짜 사진을 torch.zeros(1, 3, 160, 160)으로 바꾸고 실행하세요. layer3, layer4, avgpool, fc를 지난 모양을 먼저 예상하세요. fc는 여전히 동작할까요?

```
x = torch.zeros(1, 3, 160, 160)
```

힌트: 합성곱 몸통은 크기를 1/32로 줄인다: 224 → 7

2 **머리를 3클래스로** [b03] 기본

사과·배·귤 3가지를 구분하려고 모델.fc = nn.Linear(512, 3)으로 바꿨습니다. 새 머리의 파라미터 수와 모델 전체 파라미터 수는?

힌트: 원래 전체 11,689,512 = 몸통 11,176,512 + 머리 513,000

3 **float16으로 줄이면** [b01] 응용

b01의 모델을 모델.half()로 바꾼 뒤 바이트를 다시 계산하면 몇 MB가 될까요? element_size()가 무엇을 돌려주는지 생각해 보세요.

```
모델 = models.resnet18(weights=None).half()
```

힌트: float32 = 4바이트, float16 = 2바이트

4 **구간이 겹치면** [전이도구] 응용

누군가 학습 사진을 늘리려고 유로샛_불러오기("학습", 1200)을 썼습니다. 검증은 그대로 ("검증", 100)입니다. 어떤 문제가 생길까요? 클래스마다 섞은 순서의 몇 번째부터 몇 번째까지를 쓰는지로 설명하세요.

힌트: 구간_시작 = {"학습": 0, "검증": 1000, "시험": 1500}

먼저 예상해서 적고, 코드를 고쳐 실행해 확인 · 정답은 다음 장부터

정답 21-1. layer4 (1, 512, 5, 5), avgpool 뒤 (1, 512, 1, 1)

```
layer3   → (1, 256, 10, 10)
layer4   → (1, 512, 5, 5)
avgpool  → (1, 512, 1, 1)
fc       → (1, 1000)
```

설명

- 160 ÷ 32 = 5 → layer4 지도는 5×5. 224면 7×7, 112면 4×4(직접 실행 3번)
- 전역 평균 풀링이 크기와 상관없이 512개로 만들어 주므로 fc는 그대로 동작한다
- 9장에서 배운 전역 평균 풀링의 장점이 여기서 쓰인다: 입력 크기를 바꿔도 머리를 고칠 필요가 없다

정답 21-2. 머리 1,539개, 전체 11,178,051개

```
모델.fc = nn.Linear(512, 3)
# 머리: 512 × 3 + 3 = 1,539
# 전체: 11,176,512 + 1,539 = 11,178,051
```

설명

- 몸통(11,176,512개)은 그대로, 머리만 바뀐다
- 확인 문제(10클래스)라면 512 × 10 + 10 = 5,130개
- 머리는 전체의 0.01%대. 전이 학습에서 새로 배울 양이 얼마나 적은지 보여 준다

정답 21-3. 약 22.3MB (절반)

```
모델 = models.resnet18(weights=None).half()
바이트 = sum(p.numel() * p.element_size()
             for p in 모델.parameters())
print(바이트 / 1024**2)     # 22.30
```

설명

- 파라미터 수(1,170만)는 같고, 숫자 하나가 4바이트 → 2바이트가 됐다
- 모델 경량화(책 19~20장)의 가장 쉬운 방법이 숫자 정밀도 낮추기다
- weights=None으로도 크기는 계산된다. 정확도까지 보려면 학습된 가중치가 필요

정답 21-4. 학습 0~1199번째와 검증 1000~1099번째가 겹친다

```
# 학습: 섞기[0 : 0 + 1200]     → 0 ~ 1199번째
# 검증: 섞기[1000 : 1000 + 100] → 1000 ~ 1099번째
# → 검증 100장이 모두 학습에도 들어감
```

설명

- 같은 씨앗(k)으로 섞으니 순서가 똑같다. 검증 사진을 학습에서 이미 본 셈
- 검증 점수가 실제보다 높게 나온다 (데이터 누수, 11차시)
- 학습을 늘리려면 구간_시작도 함께 옮겨야 한다. 클래스당 최소 2,000장이니 예: 학습 0, 검증 1500, 시험 1750

22

특징 추출: 눈은 빌리고 판단만 새로

b08 ~ b12 · 책 10.4

b08 몸통 얼리기: 학습 대상은 0.046%

```
모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
for p in 모델.parameters():
    p.requires_grad_(False)                    #몸통 전체를 얼린다
모델.fc = nn.Linear(모델.fc.in_features, 10)   # 새 층은 얼어 있지 않다

전체 = sum(p.numel() for p in 모델.parameters())
학습_대상 = sum(p.numel() for p in 모델.parameters()
                if p.requires_grad)

x = torch.randn(4, 3, 112, 112)                #가짜 배치로 backward
nn.functional.cross_entropy(모델(x),
                            torch.tensor([0, 1, 2, 3])).backward()
print("conv1.weight.grad:", 모델.conv1.weight.grad)
옵티마이저 = torch.optim.Adam(모델.fc.parameters(), lr=1e-3)
```

```
전체 11,181,642개 중 학습 대상 5,130개 (0.046%)
학습 대상 이름: ['fc.weight', 'fc.bias']
conv1.weight.grad: None
fc.weight.grad 모양: (10, 512)
옵티마이저가 맡은 텐서 수: 2
```

얼린다 = requires_grad False

기울기를 계산하지 않아 .grad가 None
→ step()을 해도 바뀌지 않는다 (6장 6.4)

순서가 중요

얼리고 → 그다음 머리를 바꾼다
거꾸로 하면 새 머리까지 얼어 아무것도 학습되지 않는다

in_features

원래 fc가 받던 특징 수(512)를 그대로 받아쓴다
모델이 바뀌어도 숫자를 외울 필요 없음

b09 특징을 한 번만 뽑아 두기

```
몸통 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
몸통.fc = nn.Identity()        #마지막 층을 "그대로 통과"로 → 특징 추출기
몸통.requires_grad_(False)

학습_특징, 학습_정답 = 특징_뽑기(몸통, 로더_만들기(학습_사진, 학습_정답), 장치)
검증_특징, 검증_정답 = 특징_뽑기(몸통, 로더_만들기(검증_사진, 검증_정답), 장치)

머리 = nn.Linear(512, 10)      #이제 학습은 층 하나뿐
옵티마이저 = torch.optim.Adam(머리.parameters(), lr=1e-3)
for 에폭 in range(1, 101):
    옵티마이저.zero_grad()
    손실 = nn.functional.cross_entropy(머리(학습_특징), 학습_정답)
    손실.backward()
    옵티마이저.step()
```

```
특징: (500, 512) (1000, 512)
에폭   1: 손실 2.3806, 학습 13.8%, 검증 13.9%
에폭  10: 손실 1.6413, 학습 70.6%, 검증 61.0%
에폭  50: 손실 0.4790, 학습 94.2%, 검증 86.4%
에폭 100: 손실 0.2296, 학습 98.2%, 검증 87.0%
```

nn.Identity()

fc 자리에 "아무것도 안 하는 층"을 넣으면 모델 출력이 512개 특징이 된다

왜 뽑아 두나

매 에폭 몸통을 돌리면 500 × 100 = 50,000번
뽑아 두면 학습·검증 합쳐 1,500번 → 몇 초면 끝

대가

특징이 고정되니 데이터 증강을 쓸 수 없다 (25차시: 데이터가 아주 적을 때 미세조정과 차이가 나는 이유)

b10 얼렸는데도 바뀌는 것: 배치 정규화 기록

```
모델.requires_grad_(False)                #전부 얼렸다
모델.fc = nn.Identity()
모델.eval()
with torch.no_grad():
    전 = 모델(x)
print("bn1 기록 평균(전):", 모델.bn1.running_mean[:4])

모델.train()                              #습관대로 train 모드로 돌리면
with torch.no_grad():
    for _ in range(5):
        모델(x)
print("bn1 기록 평균(후):", 모델.bn1.running_mean[:4])

모델.eval()
with torch.no_grad():
    후 = 모델(x)
유사도 = nn.functional.cosine_similarity전, 후, dim=1)
```

```
학습 대상 파라미터: 0
bn1 기록 평균(전): ['0.003', '-0.026', '0.000', '-0.085']
bn1 기록 평균(후): ['-0.006', '0.070', '-0.000', '0.221']
같은 사진의 특징이 바뀌었나? 최대 차이 2.841, 코사인 유사도 평균 0.945
```

무엇이 바뀌었나

running_mean·running_var는 파라미터가 아니라 버퍼
기울기와 상관없이 train 모드에서 매번 갱신된다 (8장 BN)

왜 문제인가

ImageNet에서 모은 기록이 우리 배치 몇 개로 덮어써짐
→ 같은 사진인데 특징이 달라짐 (유사도 0.945)

no_grad도 소용없다

no_grad는 기울기 기록만 끈다. BN 기록 갱신은 train/eval이 정한다

b10 해결: 배치 정규화만 평가 모드로

```
def 배치정규화_고정(모델: nn.Module) -> None:
    for 층 in 모델.modules():           #모든 층을 안쪽까지
        if isinstance(층, nn.BatchNorm2d):
            층.eval()                    # BN만 평가 모드

새모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
새모델.fc = nn.Identity()
새모델.train()                          #드롭아웃 등은 학습 모드
배치정규화_고정(새모델)                 # BN만 다시 eval
with torch.no_grad():
    for _ in range(5):
        새모델(x)
print("고정한 bn1 기록 평균:", 새모델.bn1.running_mean[:4])
```

```
고정한 bn1 기록 평균: ['0.003', '-0.026', '0.000', '-0.085']   ← 그대로
```

두 스위치는 다르다

requires_grad: 기울기로 가중치를 고칠까
train()/eval(): BN 기록 갱신, 드롭아웃 켜기

순서 주의

train()은 모든 하위 층을 학습 모드로 되돌린다
→ 매 에폭 train() 뒤에 배치정규화_고정을 다시 불러야 한다 (b16이 그렇게 함)

modules()

Sequential·블록 안쪽까지 모든 층을 꺼낸다
named_children()은 바로 아래만

b11 전처리를 틀리면: 오류 없이 조용히 떨어진다

```
실수 = 학습_사진.float() / 255
우리_평균 = 실수.mean(dim=(0, 2, 3)).tolist()
우리_표준편차 = 실수.std(dim=(0, 2, 3)).tolist()

크기_맞추기 = [v2.Resize(112), v2.ToDtype(torch.float32, scale=True)]
방법들 = {
  "ImageNet 기준 정규화": v2.Compose(크기_맞추기 + [v2.Normalize(IMAGENET값)]),
  "우리 데이터 기준 정규화": v2.Compose(크기_맞추기 + [v2.Normalize(우리값)]),
  "정규화 빼먹기(0~1)": v2.Compose(크기_맞추기),
  "scale 빼먹기(0~255)": v2.Compose([v2.Resize(112),
        v2.ToDtype(torch.float32),            # scale=True가 없다
        v2.Normalize(IMAGENET값)]),
}
for 이름, 변환 in 방법들.items():   # 같은 특징 추출 실험, 전처리만 다름
    ...
```

```
검증 정확도  87.0%  ImageNet 기준 정규화
검증 정확도  84.4%  우리 데이터 기준 정규화
검증 정확도  80.1%  정규화 빼먹기(0~1)
검증 정확도  43.1%  scale 빼먹기(0~255)
```

왜 ImageNet 값이 이기나

몸통은 ImageNet 눈금으로 정규화된 입력만 보고 배웠다
우리 데이터 눈금은 몸통이 모르는 눈금

scale 빼먹기가 최악

0~255 그대로 Normalize → 값이 수백~1,000 넘게 커짐
몸통이 본 적 없는 크기 → 43.1%

교훈

코드는 오류 없이 돈다. 정확도만 조용히 떨어진다 → 숫자로 확인하는 습관

b12 입력 크기: 정확도와 계산량의 저울질

```
from torch.utils.flop_counter import FlopCounterMode

몸통.eval()              # 세는 동안에도 BN 기록이 바뀌지 않게
for 크기 in [64, 112, 160, 224]:
    with FlopCounterMode(display=False) as계수기:
        몸통.cpu()(torch.zeros(1, 3,크기, 크기))    # 사진 한 장
    몸통.to(장치)
    학습_특징, _ = 특징_뽑기(몸통, 로더_만들기(..., 크기=크기), 장치)
    검증_특징, _ = 특징_뽑기(몸통, 로더_만들기(..., 크기=크기), 장치)
    머리 = 머리_학습(학습_특징, 학습_정답)
    ...
    print(f"{크기} × {크기}: 한 장에 "
          f"{계수기.get_total_flops() / 1e9:5.2f} GFLOP,검증 {정확도:.1%}")
```

```
64 × 64: 한 장에 0.30 GFLOP, 검증 정확도 84.3%   (layer4 지도 2×2)
112 × 112: 한 장에 0.97 GFLOP, 검증 정확도 87.0%  (4×4)
160 × 160: 한 장에 1.85 GFLOP, 검증 정확도 89.1%  (5×5)
224 × 224: 한 장에 3.63 GFLOP, 검증 정확도 89.3%  (7×7)
```

FlopCounterMode

with 블록 안의 계산에서 곱셈·덧셈 횟수를 센다
GFLOP = 10억 번 단위

계산량은 넓이에 비례

크기 2배 → 칸 수 4배 → 계산량 약 4배
112 → 224: 3.7배

고르는 법

112 → 224로 키우면 +2.3%p, 계산 3.7배
실험 많이 돌릴 땐 112, 최종 모델은 160~224

22차시 문제

1 **순서를 거꾸로 하면** [b08] 기본

b08에서 순서를 바꿔, 먼저 모델.fc = nn.Linear(512, 10)으로 바꾸고 그다음 모든 파라미터를 얼렸습니다. 학습 대상 수는? backward()를 하면 어떻게 될까요?

```
모델.fc = nn.Linear(모델.fc.in_features, 10)  for p in 모델.parameters():  p.requires_grad_(False)
```

힌트: 새 머리도 모델.parameters() 안에 들어 있다

2 **layer4만 녹이기** [b08] 응용

b08처럼 전부 얼리고 머리를 바꾼 뒤, layer4만 다시 학습 대상으로 만들고 싶습니다. 코드 한 줄을 추가하고, 학습 대상 수를 예상하세요.

힌트: b03 출력: layer4 8,393,728개, 새 머리 5,130개

3 **BN 기록은 몇 개?** [b10] 응용

ResNet18에 BatchNorm2d 층은 몇 개일까요? 버퍼(기울기로 고치지 않는 기록)는 텐서 몇 개일까요? modules()와 buffers()로 세는 코드를 쓰세요.

힌트: BN 층 하나에 running_mean, running_var, num_batches_tracked

4 **scale을 빼먹으면 값이 얼마?** [b11] 기본

빨강 채널 값이 255(가장 밝음)인 픽셀이 있습니다. ImageNet 정규화(평균 0.485, 표준편차 0.229)를 거치면 얼마가 될까요? ① scale=True일 때 ② scale을 빼먹었을 때

힌트: 정규화 = (값 − 평균) / 표준편차

5 **96×96이면 계산량은?** [b12] 응용

b12의 크기 목록에 96을 넣으면 사진 한 장에 몇 GFLOP일까요? 112일 때 0.97 GFLOP를 기준으로 넓이 비율로 먼저 어림하고, 실행해 비교하세요.

```
for 크기 in [64, 96, 112, 160, 224]:
```

힌트: 계산량 ≈ 칸 수에 비례 = 크기²

먼저 예상해서 적고, 코드를 고쳐 실행해 확인 · 정답은 다음 장부터

정답 22-1. 학습 대상 0개, backward에서 오류

```
# 학습 대상 0
RuntimeError: element 0 of tensors does not require
grad
              and does not have a grad_fn
```

설명

- 새 머리까지 얼어서 기울기를 구할 파라미터가 하나도 없다
- 손실이 계산 그래프와 연결되지 않아 backward()가 오류를 낸다
- 오류가 나서 오히려 다행인 경우. 일부만 잘못 얼리면 오류 없이 학습이 안 되기도 한다 → 학습 대상 수를 출력해 확인하는 습관

정답 22-2. 모델.layer4.requires_grad_(True) → 8,398,858개

```
모델.requires_grad_(False)
모델.fc = nn.Linear(512, 10)
모델.layer4.requires_grad_(True)     # 추가
# 8,393,728 + 5,130 = 8,398,858
```

설명

- 모듈.requires_grad_(True)는 그 안의 모든 파라미터를 한 번에 녹인다
- 23차시 b16 표의 "layer4 풀기" 학습 대상 수와 같다
- 이렇게 일부만 푸는 게 미세조정의 출발점이다

정답 22-3. BN 20개, 버퍼 텐서 60개

```
bn = [m for m in 모델.modules()
      if isinstance(m, nn.BatchNorm2d)]
print(len(bn))                        # 20
print(len(list(모델.buffers())))    # 60
print([n for n, _ in 모델.named_buffers()][:3])
# ['bn1.running_mean', 'bn1.running_var',
#  'bn1.num_batches_tracked']
```

설명

- BN 층마다 버퍼가 3개씩 → 20 × 3 = 60
- 버퍼는 parameters()에 안 나오고 requires_grad도 없다. 그래서 얼려도 train() 이면 바뀐다
- state_dict()에는 버퍼도 함께 저장된다. 저장한 모델에 BN 기록까지 들어가는 이유

정답 22-4. ① 2.25 ② 1,111.4

```
# ① scale=True: 255 → 1.0
(1.0 - 0.485) / 0.229  =    2.25
# ② scale 없음: 255 그대로
(255 - 0.485) / 0.229  = 1111.4
```

설명

- 몸통은 대략 −2 ~ +2.6 범위의 입력만 보고 배웠다
- 500배 큰 값이 들어가니 특징이 엉망 → 43.1%
- 오류 메시지가 없으니, 전처리 뒤 x.mean(), x.std(), x.max()를 출력해 범위를 확인하자

정답 22-5. 약 0.67 GFLOP (어림 0.71)

```
# 어림: 0.97 × (96/112)² = 0.97 × 0.73 ≈ 0.71
# 실제: 0.666 GFLOP
```

설명

- 넓이 비례 어림이 거의 맞는다. 차이는 크기가 작을수록 가장자리·풀링 몫이 조금 달라서
- 정확도는 64(84.3%)와 112(87.0%) 사이일 것으로 예상할 수 있다 (실행해야 정확히 안다)
- 계산량은 학습된 가중치 없이도(weights=None) 셀 수 있다

23 처음부터 학습과 미세조정

b13 ~ b17 · 책 10.5 ~ 10.6

b13 처음부터 학습하면 어떨까

```
학습_로더 = 로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)
학습_평가_로더 = 로더_만들기(학습_사진, 학습_정답)  # 증강 없이 잴 때
검증_로더 = 로더_만들기(검증_사진, 검증_정답)

torch.manual_seed(0)
모델 = models.resnet18(weights=None, num_classes=10).to(장치)
#                      무작위 가중치    처음부터 머리가 10개
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-3)
for 에폭 in range(1, 31):
    손실 = 한_에폭_학습(모델, 학습_로더, 옵티마이저, 장치)
    if 에폭 % 5 == 0:
        print(에폭, 손실, 정확도(모델, 학습_평가_로더, 장치),
              정확도(모델, 검증_로더, 장치))
```

```
30에폭 뒤 검증 68.1%  (최고는 25에폭 69.9%)
학습 정확도 82.4%
→ 1,100만 개를 모두 학습했는데 특징 추출(87.0%)보다 20%p 가까이 낮다
```

같은 구조, 다른 출발점

weights=None: 무작위에서 시작
사진 500장으로 1,100만 개를 처음부터 배우기엔 데이터가 너무 적다

학습 평가용 로더를 따로

학습 정확도를 증강 없이 재야 검증과 공정하게 비교된다

왜 ImageNet이 도움이 될까

앞부분이 배운 가장자리·질감은 위성 사진에도 있는 기본 재료
뒷부분은 ImageNet 전용 → 고쳐 쓰면 더 좋다 (미세조정)

b14 미세조정: 학습률이 전부다

```
def 사전학습_모델(클래스_수=10):
    """ImageNet 가중치 + 새 머리. 아무것도 얼리지 않는다."""
    모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    모델.fc = nn.Linear(모델.fc.in_features, 클래스_수)
    return 모델

for 학습률 in [1e-2, 1e-3, 1e-4]:
    torch.manual_seed(0)
    모델 = 사전학습_모델().to(장치)
    옵티마이저 = torch.optim.Adam(모델.parameters(), lr=학습률)
    기록 = []
    for 에폭 in range(8):
        한_에폭_학습(모델, 학습_로더, 옵티마이저, 장치)
        기록.append(정확도(모델, 검증_로더, 장치))
    print(f"lr={학습률:g}:", " ".join(f"{a:.0%}" for a in 기록))
```

```
lr=0.01:   10% 17% 10% 16% 15% 23% 26% 22%
lr=0.001:  45% 50% 66% 65% 81% 82% 84% 85%
lr=0.0001: 60% 85% 89% 91% 92% 92% 93% 93%
```

미세조정 = 얼리지 않고 전부 학습

출발점이 ImageNet 가중치라는 점만 처음부터 학습과 다르다

학습률에 따라

0.01: 큰 걸음으로 물려받은 눈을 거의 다 잊음
0.001: 처음에 흐려 놓고 다시 배우느라 85%
0.0001: 둘째 에폭 85%, 끝내 93%

요령

미세조정 학습률은 처음부터 학습할 때(1e-3)보다 10배쯤 작게

b15 층마다 다른 학습률: param_groups

```
몸통_파라미터 = [p for 이름, p in 모델.named_parameters()
                 if not 이름.startswith("fc.")]
옵티마이저 = torch.optim.Adam([
    {"params": 몸통_파라미터, "lr": 1e-4},        # 잘 배운 부분은 조심스럽게
    {"params": 모델.fc.parameters(), "lr": 1e-3}, #처음 배우는 머리는 크게
])
for i, 그룹 in enumerate(옵티마이저.param_groups):
    개수 = sum(p.numel() for p in 그룹["params"])
    print(f"그룹 {i}: 텐서 {len(그룹['params'])}개, "
          f"파라미터 {개수:,}개, lr={그룹['lr']:g}")
```

```
그룹 0: 텐서 60개, 파라미터 11,176,512개, lr=0.0001
그룹 1: 텐서 2개, 파라미터 5,130개, lr=0.001
전부 1e-4:            61.7% 84.0% 89.4% 90.6% 92.2% 92.7% 92.8% 92.6%
몸통 1e-4 + 머리 1e-3: 66.7% 85.1% 87.7% 90.4% 91.8% 92.8% 93.2% 93.4%
```

사전(dict) 목록으로 넘기기

{"params": 텐서들, "lr": 값} 하나가 그룹 하나
그룹마다 학습률이 따로

named_parameters()

("layer1.0.conv1.weight", 텐서) 처럼 이름과 함께
startswith("fc.")로 머리만 골라냄

결과

첫 에폭부터 앞서고 8에폭 뒤 0.8%p 차이
머리는 무작위에서 시작하니 나누는 편이 자연스럽다

b16 어디까지 풀까

```
순서 = ["conv1", "bn1", "layer1", "layer2", "layer3", "layer4"]  #입력 쪽부터
for 풀_개수 in [0, 1, 2, 4, 6]:                #뒤에서부터 몇 부분을 풀까
    얼린_부분 = 순서[:len(순서) - 풀_개수]
    풀_부분 = 순서[len(순서) - 풀_개수:]
    모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
    for 이름 in 얼린_부분:
        모델.get_submodule(이름).requires_grad_(False)
    모델.fc = nn.Linear(512, 10)
    ...                                          #몸통 1e-4, 머리 1e-3
    for 에폭 in range(8):
        모델.train()
        for 이름 in 얼린_부분:                  # 얼린 부분은 BN 기록도 그대로
            모델.get_submodule(이름).eval()
        for x, y in 학습_로더:
            ...                                  #표준 학습 한 걸음
```

푼 부분	학습 대상	검증(8에폭)
없음 (머리만)	5,130	83.6%
layer4	8,398,858	91.4%
layer3, layer4	10,498,570	92.7%
layer1 ~ layer4	11,172,106	93.1%
전부 (conv1, bn1까지)	11,181,642	93.1%

get_submodule("layer4")

이름 문자열로 부품을 꺼낸다
"layer4.1.conv2"처럼 점으로 안쪽까지도 가능

b10 교훈 적용

train() 직후 얼린 부분만 eval()로 되돌린다

결과

layer4 하나만 풀어도 8%p 가까이 뛴다
앞부분까지 풀어도 거의 같다 → 계산이 빠듯하면 뒷부분만

b17 무엇이 얼마나 바뀌었을까

```
처음 = copy.deepcopy(모델.state_dict())    # 미세조정 전 가중치를 복사
옵티마이저 = torch.optim.Adam(모델.parameters(), lr=1e-4)
for 에폭 in range(8):
    한_에폭_학습(모델, 학습_로더, 옵티마이저, 장치)

나중 = {k: v.cpu() for k, v in 모델.state_dict().items()}
for 부분 in ["conv1", "layer1", "layer2", "layer3", "layer4"]:
    이름들 = [k for k in 처음
              if k.startswith(부분 + ".") and k.endswith("weight")]
    전 = torch.cat([처음[k].flatten() for k in 이름들])
    후 = torch.cat([나중[k].flatten() for k in 이름들])
    print(f"{부분:7} 변화율 {(후 - 전).norm() / 전.norm():.2%}")
```

```
검증 정확도 92.6%
conv1   변화율 0.55%
...
layer4  변화율 4.42%     (뒤로 갈수록 약 8배)
```

deepcopy가 필요한 이유

state_dict()는 모델 텐서를 가리키기만 한다
복사해 두지 않으면 학습하면서 같이 바뀐다 (8장 조기 종료)

변화율 = ‖나중 − 처음‖ / ‖처음‖

가중치 전체를 한 줄로 이어 붙인 벡터의 길이로 비교

해석

앞부분의 기본 특징은 위성 사진에도 맞아 거의 안 바뀜
가장 많이 바뀐 layer4도 4.42% → 물려받은 가중치는 거의 그대로

23차시 문제

1 **사전학습 + num_classes=10?** [b13] 기본

b13처럼 머리를 처음부터 10개로 만들고 싶어서 아래처럼 썼습니다. 어떻게 될까요? 그래서 사전학습 모델은 어떻게 머리를 바꾸나요?

```
모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT,   num_classes=10)
```

힌트: 가중치 파일 안의 fc는 1000개짜리다

2 **학습률을 세 그룹으로** [b15] 응용

몸통을 둘로 나눠 layer4만 5e-4로 조금 더 크게 고치려 합니다. 나머지 몸통 1e-4, layer4 5e-4, 머리 1e-3의 세 그룹을 만드세요. 그룹마다 텐서 수와 파라미터 수를 예상하세요.

힌트: startswith에 튜플을 넣으면 여러 접두어를 한 번에 확인

3 **layer2부터 풀면** [b16] 기본

b16의 풀_개수 목록에 3을 넣으면 어느 부분이 풀리고, 학습 대상은 몇 개일까요? b03의 부분별 파라미터 수로 계산하세요.

```
for 풀_개수 in [0, 1, 2, 3, 4, 6]:
```

힌트: layer2 525,568 · layer3 2,099,712 · layer4 8,393,728 · 머리 5,130

4 **변화율 손계산** [b17] 기본

어떤 층의 가중치가 미세조정 전 [3, 4]에서 후 [3, 5]로 바뀌었습니다. b17의 변화율 (후 − 전).norm() / 전.norm()은 몇 %일까요?

힌트: norm = 원소 제곱합의 루트

먼저 예상해서 적고, 코드를 고쳐 실행해 확인 · 정답은 다음 장부터

정답 23-1. ValueError: num_classes는 1000이어야 한다

```
ValueError: The parameter 'num_classes' expected value 1000
            but got 10 instead.

# 올바른 방법: 1000개짜리로 불러온 뒤 머리를 갈아 끼운다
모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
모델.fc = nn.Linear(모델.fc.in_features, 10)
```

설명

- 학습된 가중치는 1000개 출력 모양으로 저장돼 있어 10개짜리 설계도에 넣을 수 없다
- num_classes=10은 weights=None(처음부터 학습)일 때만 쓴다
- 그래서 전이 학습은 "불러오고 → 머리 바꾸기" 두 단계다

정답 23-2. 45개 / 15개 / 2개 텐서

```
l4 = [p for n, p in 모델.named_parameters() if n.startswith("layer4.")]
나머지 = [p for n, p in 모델.named_parameters()
          if not n.startswith(("layer4.", "fc."))]
옵티마이저 = torch.optim.Adam([
    {"params": 나머지, "lr": 1e-4},           # 45개, 2,782,784
    {"params": l4, "lr": 5e-4},               # 15개, 8,393,728
    {"params": 모델.fc.parameters(), "lr": 1e-3}])  # 2개, 5,130
```

설명

- 원래 몸통 60개 텐서 = 나머지 45 + layer4 15
- 뒤로 갈수록 학습률을 크게 주는 걸 "층별 학습률(discriminative lr)"이라고 한다
- 효과는 검증으로 확인해야 한다. b16·b17 결과(뒷부분이 더 바뀌어야 함)와 같은 방향의 아이디어

정답 23-3. layer2, layer3, layer4 → 11,024,138개

```
# 순서[6 - 3:] = ["layer2", "layer3", "layer4"]
# 525,568 + 2,099,712 + 8,393,728 + 5,130
#   = 11,024,138
```

설명

- 뒤에서부터 3부분이 풀린다
- 정확도는 layer3·4(92.7%)와 layer1~4(93.1%) 사이일 것으로 예상
- layer2는 파라미터가 적어서 풀어도 계산 부담이 크게 늘지 않는다

정답 23–4. 20%

```
전 = [3, 4]   →  ‖전‖ = √(9 + 16) = 5
후 − 전 = [0, 1] →  ‖후 − 전‖ = 1
변화율 = 1 / 5 = 0.20 = 20%
```

설명

- 원래 크기에 비해 얼마나 움직였나를 비율로 본다
- 실제 layer4는 4.42%: 숫자 수백만 개가 평균적으로 원래 크기의 몇 %만 움직였다
- 8장 그래디언트 클리핑에서 쓴 norm(길이)과 같은 계산

24 다른 모델, 다른 입력, 저장

b18 · b19 · b23 · b20 · 책 10.7 ~ 10.8

b18 머리 바꾸기를 일반화하기

```
def 머리_바꾸기(모델: nn.Module, 클래스_수: int) -> str:
    """마지막 nn.Linear를 찾아 클래스_수만큼 내는 새 층으로 바꾼다."""
    이름, 마지막 = [(n, m) for n, m in 모델.named_modules()
                    if isinstance(m, nn.Linear)][-1]          # ① 찾기
    부모_이름, _, 자식_이름 = 이름.rpartition(".")            # ② 나누기
    # "classifier.3" → "classifier", ".", "3"
    setattr(모델.get_submodule(부모_이름), 자식_이름,         # ③ 갈아 끼우기
            nn.Linear(마지막.in_features, 클래스_수))
    return 이름

for 이름 in ["resnet18", "mobilenet_v3_small", "efficientnet_b0", "vit_b_16"]:
    모델 = models.get_model(이름, weights=None)     # 구조만
    바꾼_곳 = 머리_바꾸기(모델, 10)
```

```
resnet18            마지막 층 'fc'          ← 512    → 출력 (1, 10)
mobilenet_v3_small  마지막 층 'classifier.3' ← 1,024 → 출력 (1, 10)
efficientnet_b0     마지막 층 'classifier.1' ← 1,280 → 출력 (1, 10)
vit_b_16            마지막 층 'heads.head'   ← 768   → 출력 (1, 10)
```

모델마다 머리 위치가 다르다

resnet은 fc, mobilenet은 classifier.3 … 모델.fc = … 한 줄로는 안 된다

rpartition(".")

마지막 점을 기준으로 셋으로 나눔
점이 없으면 ("", "", "fc") → 부모는 모델 자신

setattr(부모, 이름, 새것)

부모.이름 = 새것 과 같다. 이름이 문자열일 때 쓴다
처음 쓰는 모델은 print(모델)로 한 번 훑어보기

b19 몸통 바꿔 끼우기: 순위가 뒤집힌다

```
for 이름 in ["mobilenet_v3_small", "resnet18", "efficientnet_b0"]:
    모델 = models.get_model(이름, weights="DEFAULT")     # 가중치 다운로드
    마지막 = [n for n, m in 모델.named_modules()
              if isinstance(m, nn.Linear)][-1]
    부모, _, 자식 = 마지막.rpartition(".")
    setattr(모델.get_submodule(부모), 자식, nn.Identity())   # 머리 떼기
    학습_특징, _ = 특징_뽑기(모델, 로더_만들기(학습_사진, 학습_정답), 장치)
    검증_특징, _ = 특징_뽑기(모델, 로더_만들기(검증_사진, 검증_정답), 장치)
    머리 = 머리_학습(학습_특징, 학습_정답)
    ...
```

모델	몸통	특징	ImageNet top-1	우리 검증
MobileNetV3-Small	150만	1,024	67.7%	85.5%
ResNet18	1,120만	512	69.8%	87.0%
EfficientNet-B0	400만	1,280	77.7%	85.6%

같은 함수, 다른 몸통

b18의 머리 찾기를 그대로 써서 마지막 층을 Identity로 바꾸면 어떤 모델이든 특징 추출기가 된다

순위가 뒤집혔다

ImageNet 1등 EfficientNet이 우리 데이터에선 ResNet18보다 낮다
→ 모델 고르기도 우리 검증 데이터로

작은 기기라면

MobileNet은 몸통이 7분의 1인데 차이는 1.5%p

b23 흑백 사진을 넣고 싶다면

```
모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT).eval()
print(tuple(모델.conv1.weight.shape))    # 출력 64, 입력 3색, 7, 7)
흑백 = torch.randn(2, 1, 64, 64)          #흑백 사진 2장 (예제용 난수)

# 방법 1: 한 채널을 세 번 복사해 컬러처럼
with torch.no_grad():
    출력1 = 모델(흑백.repeat(1, 3, 1, 1))    # (2,1,64,64) → (2,3,64,64)

# 방법 2: 첫 층을 1채널로 바꾸고 세 색의 가중치를 더해 넣기
새_첫층 = nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3, bias=False)
with torch.no_grad():
    새_첫층.weight.copy_(모델.conv1.weight.sum(dim=1, keepdim=True))
모델.conv1 = 새_첫층                          # (64, 3, 7, 7) → (64, 1, 7, 7)
with torch.no_grad():
    출력2 = 모델(흑백)
print(torch.allclose(출력1, 출력2, atol=1e-4))
```

```
conv1 가중치: (64, 3, 7, 7)
conv1 가중치: (64, 1, 7, 7)
두 방법의 출력이 같은가: True
```

왜 같을까

세 채널 값이 모두 a라면
a×w1 + a×w2 + a×w3 = a×(w1 + w2 + w3)
복사해서 각각 곱하나, 가중치를 더해 한 번 곱하나 같다

repeat(1, 3, 1, 1)

차원마다 몇 번 반복할지: 채널만 3번

실무에서는

대부분 방법 1로 충분하다
어느 쪽이든 정규화는 세 채널로 만든 뒤 ImageNet 값으로

b20 저장하기: 전체냐 머리만이냐

```
장치 = torch.device("cpu")
모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
모델.fc = nn.Identity()
학습_특징, _ = 특징_뽑기(모델, 로더_만들기(학습_사진, 학습_정답), 장치)
모델.fc = 머리_학습(학습_특징, 학습_정답)      # 완성 모델 (검증 87.0%)

torch.save(모델.state_dict(), "eurosat_resnet18.pt")    #모델 전체
torch.save(모델.fc.state_dict(), "eurosat_head.pt")     #머리만
for 파일 in ["eurosat_resnet18.pt", "eurosat_head.pt"]:
    print(f"{파일:20} {Path(파일).stat().st_size / 1024:10,.1f}KB")
```

```
완성 모델 검증 정확도 87.0%
eurosat_resnet18.pt    43,755.1KB
eurosat_head.pt            21.9KB      ← 2,000배 차이
```

state_dict()

층 이름 → 텐서 사전. 설계도는 저장되지 않는다 (6장)
모델.fc.state_dict()는 fc 부분만

머리만 저장해도 될 때

특징 추출: 몸통은 안 바뀌었고 torchvision에서 언제든 다시 받을 수 있다

전체를 저장해야 할 때

미세조정: 몸통도 바뀌었으니 전체가 필요하다

b20 불러오기: 같은 설계도부터

```
# ① 설계도를 그대로 만들지 않으면
새모델 = models.resnet18(weights=None)          # fc 가 1000개짜리
try:
    새모델.load_state_dict(torch.load("eurosat_resnet18.pt", weights_only=True))
except RuntimeError as e:
    ...                                          #  오류 출력

# ② 머리를 바꾼 설계도에 전체를 채운다
새모델.fc = nn.Linear(512, 10)
새모델.load_state_dict(torch.load("eurosat_resnet18.pt", weights_only=True))

# ③ 몸통은 torchvision에서 받고 머리만 채운다
가벼운모델 = models.resnet18(weights=models.ResNet18_Weights.DEFAULT)
가벼운모델.fc = nn.Linear(512, 10)
가벼운모델.fc.load_state_dict(torch.load("eurosat_head.pt", weights_only=True))
```

```
오류: size mismatch for fc.weight: copying a param with shape
      torch.Size([10, 512]) from checkpoint,
      the shape in current model is torch.Size([1000, 512]).
전체를 불러온 모델 87.0%
머리만 불러온 모델 87.0%
```

① 오류 읽기

저장된 것(checkpoint)은 [10, 512], 지금 설계도는 [1000, 512]
→ 머리를 바꾼 설계도를 똑같이 만들어야 한다

②와 ③ 결과가 같다

특징 추출이라 몸통이 ImageNet 가중치 그대로였기 때문

weights_only=True

숫자(텐서)만 읽고 파일 속 코드는 실행하지 않는다 (안전)

24차시 문제

1 **흑백 방법 손계산** [b23] 기본

흑백 픽셀 값 a = 2, 한 자리의 세 색 가중치가 w = [0.1, 0.2, 0.3]입니다. 방법 1(세 번 복사)과 방법 2(가중치를 더함)의 계산 결과를 각각 구하세요.

힌트: 방법 1: 각 채널에 a를 넣고 곱해 더함 / 방법 2: 가중치를 먼저 더함

2 **4채널(RGB + 적외선) 입력** [b23] 응용

위성 사진에 적외선 채널이 하나 더 있어 입력이 4채널입니다. conv1을 4채널짜리로 바꾸세요. 앞 3채널은 원래 가중치를, 4번째 채널은 세 색 가중치의 평균을 넣습니다.

힌트: torch.cat([...], dim=1), .mean(dim=1, keepdim=True)

3 **AlexNet과 VGG의 머리는 어디?** [b18] 기본

b18의 모델 목록을 ["alexnet", "vgg16"]으로 바꾸면 머리_바꾸기가 찾는 마지막 층 이름과 in_features는? (weights=None이라 다운로드 없이 바로 돈다)

```
for 이름 in ["alexnet", "vgg16"]:
```

힌트: VGG 끝의 완전 연결 4096 → 4096 → 1000 (9.8절)

4 **100클래스 머리 파일 크기** [b20] 응용

10클래스 머리 파일(eurosat_head.pt)은 21.9KB였습니다. 같은 몸통에 100클래스 머리 nn.Linear(512, 100)를 저장하면 몇 KB쯤일까요? 파라미터 수 × 4바이트로 어림하세요.

힌트: 512 × 클래스 + 클래스

정답 24-1. 둘 다 1.2

```
# 방법 1: 2×0.1 + 2×0.2 + 2×0.3 = 0.2 + 0.4 + 0.6 =
1.2
# 방법 2: 2 × (0.1 + 0.2 + 0.3) = 2 × 0.6 = 1.2
```

설명

- 분배 법칙 a×w1 + a×w2 + a×w3 = a×(w1 + w2 + w3) 그대로
- 7×7 칸 49개, 출력 채널 64개 모두에서 같은 일이 일어나 결과가 같다
- 방법 2는 첫 층 계산이 1/3로 준다. 방법 1은 코드 수정이 없다

정답 24-2. conv1 가중치 (64, 4, 7, 7)

```
w = 모델.conv1.weight                              # (64, 3, 7, 7)
새_첫층 = nn.Conv2d(4, 64, kernel_size=7, stride=2,
                    padding=3, bias=False)
with torch.no_grad():
    새_첫층.weight.copy_(torch.cat(
        [w, w.mean(dim=1, keepdim=True)], dim=1))    # (64, 4, 7, 7)
모델.conv1 = 새_첫층
print(모델(torch.randn(2, 4, 64, 64)).shape)      # (2, 1000)
```

설명

- 채널 차원(dim=1)으로 이어 붙여 3 + 1 = 4채널
- 4번째 채널은 학습한 적이 없으니 평균값으로 무난하게 시작하고, 미세조정으로 고쳐 간다
- 정규화도 4채널용 평균·표준편차가 필요하다 (적외선 채널은 우리 데이터로 구함)

정답 24-3. 둘 다 'classifier.6', in_features 4096

```
alexnet   마지막 층 'classifier.6' ← 4096 → 출력 (1, 10)
vgg16     마지막 층 'classifier.6' ← 4096 → 출력 (1, 10)
```

설명

- classifier Sequential 안에 Dropout·Linear·ReLU가 섞여 있어 마지막 Linear가 6번 자리
- rpartition으로 "classifier"와 "6"으로 나눠 setattr
- 머리 위치를 몰라도 함수 하나로 처리된다. 이것이 일반화의 이점

정답 24-4. 약 200KB

```
# 10클래스:  5,130개 × 4바이트 =  20,520바이트 ≈ 20KB (+ 파일 머리말)
# 100클래스: 51,300개 × 4바이트 = 205,200바이트 ≈ 200KB
# 실제 저장: 22,277바이트(21.8KB) / 206,989바이트(202KB)
```

설명

- 파일 크기는 파라미터 수 × 4바이트 + 조금의 머리말(이름 등)
- 100클래스여도 전체 모델(43MB)보다 200배 이상 작다
- 머리만 저장하는 방식은 클래스가 많아져도 여전히 가볍다

25 실습: 데이터 양별 비교

데이터가 적을수록 전이 학습이 빛난다 · b21 · b22 · 책 10.9

b21 정해진 횟수만큼 고치기

```
def 끝없이(로더):
    """다 돌면 처음부터 다시. 돌 때마다 새로 섞고 새로 증강."""
    while True:
        yield from 로더

def 고치기(모델, 옵티마이저, 학습_로더, 횟수):
    """데이터 양과 상관없이 정해진 횟수만큼 배치를 꺼내 고친다."""
    모델.train()
    for x, y in itertools.islice(끝없이(학습_로더), 횟수):
        x, y = x.to(장치), y.to(장치)
        옵티마이저.zero_grad()
        nn.functional.cross_entropy(모델(x), y).backward()
        옵티마이저.step()
```

```
클래스당 5장이면 한 에폭이 배치 1개, 100장이면 16개
→ 에폭이 아니라 "배치를 꺼내 고치는 횟수"를 맞춰야 공정하다
처음부터 240번 · 미세조정 64번 · 특징 추출은 머리 100에폭
```

yield from

로더가 내주는 배치를 하나씩 그대로 내보낸다
while True로 감싸면 끝없이 이어짐 (제너레이터)

itertools.islice(…, 횟수)

끝없는 흐름에서 앞의 횟수만큼만 잘라 쓴다

itertools.cycle은 함정

첫 바퀴 배치를 저장해 똑같이 되풀이한다
→ 섞기도 증강도 첫 바퀴뿐

b21 세 방법을 한 표로

```
def 처음부터(학습_로더):
    모델 = models.resnet18(weights=None, num_classes=10).to(장치)
    고치기(모델, torch.optim.Adam(모델.parameters(), lr=1e-3), 학습_로더, 240)
    return 정확도(모델, 검증_로더, 장치)

def 특징_추출(학습_사진, 학습_정답):    # 몸통 얼리고 특징 → 머리_학습
    ...

def 미세조정(학습_로더):                 # 몸통 1e-4 + 머리 1e-3, 64번
    ...

for 클래스당 in [5, 20, 100]:
    학습_사진, 학습_정답 = 유로샛_불러오기("학습", 클래스당)
    학습_로더 = 로더_만들기(학습_사진, 학습_정답, 증강=True, 섞기=True)
    결과 = [처음부터(학습_로더), 특징_추출(학습_사진, 학습_정답),
            미세조정(학습_로더)]
```

```
클래스당 | 처음부터 | 특징 추출 | 미세조정
    5장 |   51.7% |   72.0% |  81.5%
   20장 |   50.6% |   83.4% |  85.9%
  100장 |   52.7% |   88.0% |  93.2%
```

데이터가 적을수록 차이가 크다

사진 50장(클래스당 5장)으로도 미세조정 81.5%
5장에서 미세조정이 특징 추출보다 9.5%p 앞섬 (특징 추출은 증강 불가)

처음부터는 왜 50% 초반?

데이터가 20배 늘어도 고치는 횟수는 240번 그대로
무작위 출발은 끝나는 순간에 따라 크게 흔들린다

실무 순서

① 특징 추출로 빠르게 기준을 잰다
② 시간이 허락하면 미세조정으로 더 올린다

b22 시험 데이터로 마지막 확인

```
학습_사진, 학습_정답 = 유로샛_불러오기("학습", 100)
시험_사진, 시험_정답 = 유로샛_불러오기("시험", 100)    # 처음이자 마지막
...                                  #미세조정: 몸통 1e-4 + 머리 1e-3
for 에폭 in range(4):                # 1,00장 ÷ 64 = 16배치 × 4 = 64번
    한_에폭_학습(모델, 학습_로더, 옵티마이저, 장치)
print(f"시험 정확도 {정확도(모델, 로더_만들기(시험_사진, 시험_정답), 장치):.1%}")

모델.eval()
with torch.no_grad():
    예측 = torch.cat([모델(x.to(장치)).argmax(1).cpu()
                      for x, _ in로더_만들기(시험_사진, 시험_정답)])
for k in range(10):                  #클래스별 정확도
    print(이름[k], (예측[시험_정답 == k] == k).float().mean())
```

```
시험 정확도 92.9%      (검증 93.2%와 거의 같다)
경작지 90%  숲 97%  초지 87%  고속도로 93%  산업지역 99%
목초지 92%  과수원 89%  주거지 95%  강 93%  바다·호수 94%
```

시험은 딱 한 번

방법·크기·학습률은 모두 검증으로 고르고
시험은 마지막에
시험 보고 고치면 시험에 맞춰 버린 셈
(11차시)

검증 ≈ 시험

93.2% vs 92.9% → 검증 성적을 믿을 수 있었다

클래스별로 보면

초지(87%)·과수원(89%)이 낮다 → 다음 장 헷갈린 짝

b22 가장 많이 헷갈린 짝

```
혼동 = torch.bincount(시험_정답 * 10 + 예측,
                      minlength=100).reshape(10, 10)    # 정답, 예측) 개수
혼동.fill_diagonal_(0)              #맞힌 것(대각선)은 지운다
for 위치 in 혼동.flatten().topk(3).indices.tolist():
    정답, 답 = divmod(위치, 10)     # 위치 = 정답×10 + 예측
    print(f"정답 {이름[정답]} → 예측 {이름[답]}: {혼동[정답, 답]}번")
```

```
정답 초지 → 예측 숲: 6번
정답 과수원 → 예측 초지: 6번
정답 초지 → 예측 과수원: 5번
```

bincount로 혼동 행렬

정답 7, 예측 2 → 번호 72
0~99번 칸의 개수를 세고 10×10으로 접으면 (정답, 예측) 표
(9장 b11의 index_put_과 같은 결과를 더 짧게)

divmod(72, 10) = (7, 2)

번호를 다시 (정답, 예측)으로 되돌린다

해석

모두 식물로 덮인 땅끼리. 64×64 한 장에서는 사람이 봐도 애매하다

25차시 문제

1 **고치는 횟수를 에폭으로 바꾸면** [b21] 기본

배치 크기 64일 때, 클래스당 5·20·100장에서 한 에폭은 배치 몇 개일까요? 처음부터(240번)와 미세조정(64번)은 각각 몇 에폭에 해당하나요?

힌트: 학습 사진 = 클래스당 × 10, 배치 수는 올림

2 **cycle의 함정 직접 보기** [b21] 응용

숫자 0~5를 3개씩, 섞어서 꺼내는 작은 로더로 itertools.cycle과 끝없이()를 비교하세요. 배치 4개를 꺼냈을 때 3·4번째 배치는 1·2번째와 같을까요?

```
로더 = DataLoader(TensorDataset(torch.arange(6)), batch_size=3, shuffle=True,
generator=torch.Generator().manual_seed(0))
```

힌트: [b[0].tolist() for b in itertools.islice(…, 4)]

3 **혼동 행렬 번호 읽기** [b22] 기본

혼동.flatten().topk(3).indices가 [21, 62, 26]을 돌려줬습니다. 각각 (정답, 예측)은? 클래스 이름으로 읽으세요. (0 경작지 · 1 숲 · 2 초지 · 6 과수원)

힌트: divmod(번호, 10) = (번호 // 10, 번호 % 10)

4 **가장 약한 클래스 두 개** [b22] 응용

b22 끝에, 클래스별 정확도가 가장 낮은 두 클래스의 이름과 정확도를 출력하는 코드를 추가하세요.

힌트: topk(2, largest=False)

먼저 예상해서 적고, 코드를 고쳐 실행해 확인 · 정답은 다음 장부터

정답 25-1. 데이터가 적을수록 에폭이 많아진다

클래스당	사진	배치/에폭	처음부터(240번)	미세조정(64번)
5장	50	1	240에폭	64에폭
20장	200	4	60에폭	16에폭
100장	1,000	16	15에폭	4에폭

설명

- ceil(사진 수 / 64): 50장도 배치 1개, 1,000장은 16개(마지막은 40장)
- 에폭 수를 맞추면 데이터가 적은 쪽이 훨씬 덜 고쳐져 불공정하다
- b22가 4에폭인 이유: 100장 × 10 = 1,000장 → 16 × 4 = 64번

정답 25-2. cycle은 반복, 끝없이는 새로 섞임

```
cycle:   [[5, 1, 0], [3, 2, 4], [5, 1, 0], [3, 2, 4]]   ← 그대로 반복
끝없이: [[5, 1, 0], [3, 2, 4], [4, 2, 1], [3, 0, 5]]   ← 새로 섞임

list(itertools.islice(itertools.cycle(로더), 4))
list(itertools.islice(끝없이(로더), 4))
```

설명

- cycle은 첫 바퀴에서 받은 값을 저장해 두고 되풀이한다. 로더를 다시 돌리지 않는다
- 증강도 마찬가지: 첫 바퀴에 만든 사진이 그대로 반복 → 증강 효과가 사라짐
- 끝없이()는 바퀴마다 for를 새로 시작하니 섞기와 증강이 매번 새로 된다

정답 25-3. 초지→숲, 과수원→초지, 초지→과수원

```
divmod(21, 10) = (2, 1)   # 정답 초지 → 예측 숲
divmod(62, 10) = (6, 2)   # 정답 과수원 → 예측 초지
divmod(26, 10) = (2, 6)   # 정답 초지 → 예측 과수원
```

설명

- b22 출력의 헷갈린 짝 세 개와 같다
- 만들 때 정답 × 10 + 예측으로 번호를 붙였으니 10으로 나눈 몫이 정답, 나머지가 예측
- 4장 bincount, 9장 혼동 행렬과 같은 아이디어

정답 25-4. 초지 87%, 과수원 89%

```
클래스별 = torch.stack([(예측[시험_정답 == k] == k).float().mean()
                        for k in range(10)])
값들, 번호들 = 클래스별.topk(2, largest=False)
for v, k in zip(값들, 번호들):
    print(이름[k], f"{v:.0%}")
```

설명

- largest=False면 작은 것부터 k개
- 헷갈린 짝(초지↔숲·과수원)과 같은 클래스가 나온다 → 데이터를 직접 들여다볼 곳
- 개선 아이디어: 입력 크기 키우기(b12), 그 클래스 사진 더 모으기. 효과는 검증으로 확인

5일차 정리

01	**불러오기**	models.resnet18(weights=DEFAULT) · meta로 정보 확인 · transforms()로 전처리 확인
02	**몸통과 머리**	몸통은 사진 → 특징 512개, 머리(fc)를 우리 클래스 수로 바꾼다
03	**특징 추출**	얼리고(requires_grad_(False)) 특징을 뽑아 두면 빠르다. BN은 eval()로 묶기
04	**미세조정**	학습률 10배 작게, 몸통 1e-4 + 머리 1e-3. 뒷부분일수록 많이 바뀐다
05	**다른 모델·입력·저장**	마지막 Linear 찾아 바꾸기 · 흑백은 3채널 복사 · 특징 추출이면 머리만 저장
06	**언제 무엇을**	데이터가 적을수록 전이 학습이 빛난다. 고르기는 검증, 시험은 한 번